

■ Trampos

Guia de Estudo — Defesa da Sprint 1 Plataforma de Vagas Freelancer

Este documento contém as perguntas mais prováveis da banca examinadora com respostas detalhadas para você demonstrar autoria e domínio técnico.

■ Item Avaliado	Status
CRUD de Usuário completo (Create/Read/Update/Delete)	■ Funcionando
Autenticação com senha criptografada (bcrypt)	■ \$2b\$12\$...
Banco de Dados MySQL integrado	■ 4 tabelas
Interface com identidade visual própria	■ CSS + Logo
Responsividade (mobile-friendly)	■ @media 640px
Validação de formulário (5+ campos)	■ 6 campos + regex
Backend FastAPI rodando na porta 8000	■ uvicorn

1. Visão Geral do Projeto

Pergunta 1 — O que é o Trampos? Qual problema ele resolve?

O **Trampos** é uma plataforma web que conecta **empresas** que precisam de profissionais para trabalhos pontuais com **freelancers** que buscam oportunidades.

O problema central é a dificuldade de pequenas empresas encontrarem mão-de-obra qualificada para tarefas específicas (eventos, reformas, entregas, aulas) sem contratar formalmente. O Trampos funciona como um marketplace de vagas avulsas.

■ Dica: Cite exemplos reais: 'uma empresa de eventos que precisa de 5 garçons para sábado'.

Pergunta 2 — Quais são os dois tipos de usuário e o que cada um faz?

O sistema possui dois perfis distintos, definidos no campo **tipo** do modelo de usuário:

- **Empresa:** pode publicar vagas (título, descrição, local, data, pagamento), gerenciar suas vagas e visualizar candidatos.

- **Freelancer:** pode navegar pelas vagas disponíveis, se candidatar a uma vaga e acompanhar o status da candidatura (pendente / aceito / recusado).

■ Dica: O tipo é gravado no banco como ENUM('empresa','freelancer') na tabela Usuario.

Pergunta 3 — Quais tecnologias foram usadas e por quê cada uma foi escolhida?

Frontend: HTML5, CSS3 puro e JavaScript vanilla — sem frameworks pesados, mantendo o projeto leve e de fácil entendimento.

Backend: Python + FastAPI — framework moderno, rápido, com validação automática via Pydantic e documentação Swagger gerada automaticamente.

Banco de dados: MySQL — banco relacional robusto, amplamente usado no mercado, ideal para relacionamentos como usuário ↔ vaga ↔ candidatura.

ORM: SQLAlchemy — abstrai o SQL, evita SQL injection e facilita migrações.

Criptografia: bcrypt (biblioteca passlib/bcrypt) — algoritmo de hash adaptativo, considerado padrão da indústria para senhas.

2. Banco de Dados MySQL

Pergunta 4 — Quais são as tabelas do banco e como elas se relacionam?

O banco **tramos** possui 4 tabelas principais:

- **Usuario** — armazena todos os usuários (empresas e freelancers). Campos: id_usuario, nome, email, senha (hash), tipo.
- **Vaga** — cada vaga pertence a uma empresa (FK id_empresa → Usuario). Campos: id_vaga, titulo, descricao, data, local, pagamento, id_empresa, id_categoria.
- **Categoria** — categorias das vagas (ex: Tecnologia, Construção). Campos: id_categoria, nome.
- **Candidatura** — tabela de junção entre Usuario e Vaga, registra quem se candidatou a qual vaga. Status: pendente, aceito, recusado.

Relacionamentos: Usuario 1→N Vaga | Usuario 1→N Candidatura | Vaga 1→N Candidatura | Categoria 1→N Vaga.

■ Dica: Desenhe mentalmente o diagrama ER: Usuario no centro, ligado a Vaga (como empresa) e a Candidatura (como freelancer).

Pergunta 5 — O que é uma chave estrangeira (FK)? Cite um exemplo do projeto.

Uma **chave estrangeira (Foreign Key)** é um campo que referencia a chave primária de outra tabela, garantindo integridade referencial — ou seja, não é possível inserir um registro que aponte para um registro inexistente.

Exemplo no projeto:

```
id_empresa INTEGER NOT NULL REFERENCES Usuario(id_usuario) ON DELETE CASCADE
```

A coluna **id_empresa** da tabela Vaga referencia **id_usuario** da tabela Usuario. O **ON DELETE CASCADE** garante que, ao deletar um usuário, todas as suas vagas também são removidas.

automaticamente.

■ Dica: `ON DELETE CASCADE` evita registros 'órfãos' no banco.

Pergunta 6 — O que é ORM? Por que usamos SQLAlchemy em vez de SQL puro?

ORM (Object-Relational Mapping) é uma técnica que mapeia tabelas do banco para classes Python, permitindo manipular dados como objetos em vez de escrever SQL manualmente.

Vantagens do SQLAlchemy no projeto:

- **Segurança:** parâmetros são automaticamente escapados, prevenindo SQL Injection.
- **Portabilidade:** trocar MySQL por PostgreSQL exige apenas mudar a string de conexão.
- **Produtividade:** queries como `db.query(models.Usuario).filter(...).first()` são mais legíveis.

Exemplo de como buscamos um usuário:

```
usuario = db.query(models.Usuario).filter(models.Usuario.email == email).first()
```

3. CRUD de Usuário

Pergunta 7 — O que significa CRUD? Mostre cada operação no projeto.

CRUD = Create, Read, Update, Delete — as 4 operações básicas de persistência de dados.

Create (POST /usuarios) — cadastra novo usuário, hasheia a senha com `bcrypt` antes de salvar.

Read (GET /usuarios e GET /usuarios/{id}) — lista todos os usuários ou busca um específico por ID.

Update (PUT /usuarios/{id}) — atualiza nome, email e/ou senha. Se nova senha for fornecida, ela é re-hasheada.

Delete (DELETE /usuarios/{id}) — remove o usuário e, por `CASCADE`, remove suas vagas e candidaturas.

O login (POST /login) não é CRUD — é autenticação, verifica credenciais sem criar dados novos.

■ Dica: Durante a defesa, abra o Swagger em <http://localhost:8000/docs> e execute cada operação ao vivo.

Pergunta 8 — O que acontece se eu tentar cadastrar um email que já existe?

O backend verifica duplicidade **antes** de inserir no banco, na rota POST /usuarios:

```
existente = db.query(models.Usuario).filter(models.Usuario.email ==  
usuario.email).first()
```

Se já existe um usuário com aquele email, retorna **HTTP 400 Bad Request** com a mensagem:

```
{"detail": "Email já cadastrado."}
```

No frontend, o toast vermelho exibe essa mensagem para o usuário.

Além disso, a coluna **email** na tabela MySQL tem a constraint **UNIQUE**, então mesmo que passasse pelo backend, o banco rejeitaria a inserção.

Pergunta 9 — Como funciona a atualização parcial do usuário (PUT)?

Usamos o schema **UsuarioUpdate** com todos os campos opcionais (Optional):

```
class UsuarioUpdate(BaseModel): nome: Optional[str] = None email: Optional[EmailStr] = None senha: Optional[str] = None
```

O método **model_dump(exclude_unset=True)** retorna apenas os campos que foram enviados na requisição, evitando sobrescrever dados com None.

Assim, se o usuário enviar apenas {"nome": "Novo Nome"}, somente o nome é alterado — email e senha permanecem intactos.

■ Dica: Isso é chamado de PATCH semântico dentro de um PUT — flexível e seguro.

4. Autenticação e Criptografia de Senha

Pergunta 10 — Por que não salvamos a senha em texto puro no banco?

Salvar senhas em texto puro é uma **vulnerabilidade crítica de segurança**. Se o banco for comprometido (vazamento, invasão), todas as senhas de todos os usuários ficam expostas.

Com **hash bcrypt**, mesmo que o invasor obtenha o banco, ele vê apenas algo como **\$2b\$12\$mHKOQ9t5TE80r...** — impossível de reverter para a senha original.

O bcrypt é especialmente seguro pois: (1) inclui **salt** aleatório automaticamente, impedindo ataques de rainbow table; (2) possui fator de custo ajustável (**rounds=12**), tornando ataques de força bruta lentos.

■ Dica: Se o professor perguntar: 'E MD5/SHA1?' — responda que são algoritmos de hash rápidos, inseguros para senhas pois permitem ataques de força bruta acelerada.

Pergunta 11 — Como é feito o processo de login? Como verificamos a senha?

O login **nunca** descriptografa a senha. O processo é:

1. Usuário envia email + senha em texto puro via POST /login.
2. Backend busca o usuário pelo email no banco.
3. A função `bcrypt.checkpw()` re-hasheia a senha fornecida e compara com o hash armazenado:

```
return bcrypt.checkpw(plain.encode(), hashed.encode())
```

4. Se baterem → retorna dados do usuário (sem a senha). Se não → HTTP 401 Unauthorized.

O frontend salva os dados do usuário no **localStorage** para manter a sessão.

■ Dica: Diga que não usamos JWT nesta sprint por simplicidade, mas seria o próximo passo para uma autenticação stateless mais robusta.

Pergunta 12 — O que é salt no bcrypt? Por que é importante?

Salt é um valor aleatório gerado automaticamente pelo bcrypt e concatenado à senha antes do hash. Cada senha gera um salt diferente.

Isso significa que dois usuários com a **mesma senha** terão **hashes diferentes** no banco:

```
senha: 123456 → $2b$12$abc...xyz (usuário A)
```

senha: 123456 → \$2b\$12\$def...uvw (usuário B)

Sem salt, um atacante poderia criar uma tabela de hashes pré-computados (rainbow table) e encontrar senhas rapidamente. Com salt, cada hash precisa ser quebrado individualmente.

5. Frontend — Interface e Validação

Pergunta 13 — Quais telas existem no frontend e o que cada uma faz?

O frontend possui **6 páginas HTML**:

- **login.html** — formulário de login com validação de email e senha.
- **register.html** — cadastro com 6 campos: nome, email, tipo (radio), senha, confirmar senha.
- **index.html** — listagem de vagas com filtro por busca e categoria.
- **job.html** — detalhe de uma vaga, com botão de candidatura (freelancer) ou lista de candidatos (empresa).
- **create-job.html** — formulário para empresa publicar nova vaga (título, local, descrição, data, pagamento, categoria).
- **profile.html** — perfil do usuário com edição de dados, exclusão de conta e atividade (vagas ou candidaturas).

Pergunta 14 — Como funciona a validação dos campos no formulário?

A validação é feita no arquivo **auth.js** usando **expressões regulares (regex)** antes de enviar ao servidor:

regex.nome: /^[a-zA-ZÀ-ÿ\s]{3,100}\$/ → apenas letras, 3-100 chars

regex.email: /^[a-zA-Z0-9._%+\-]+@.../ → formato válido de email

regex.senha: /^(.{6,})\$/ → mínimo 6 caracteres

A função **validate(input, regex, mensagem)** aplica a regra, marca o campo visualmente (borda vermelha/verde) e exibe a mensagem de erro abaixo do campo.

A confirmação de senha verifica **senha === confirmar** antes de enviar.

Além da validação frontend, o backend também valida via Pydantic (**EmailStr**, campos obrigatórios), garantindo dupla proteção.

■ Dica: A validação no frontend é para UX (resposta rápida). A do backend é a que realmente garante segurança.

Pergunta 15 — O que é responsividade? Como implementamos no projeto?

Responsividade é a capacidade do layout se adaptar a diferentes tamanhos de tela (celular, tablet, desktop) sem perder usabilidade.

Implementamos com um **@media query** no CSS para telas até 640px:

@media (max-width: 640px) { ... }

Adaptações aplicadas em mobile:

- **Navegação**: menu hamburguer (■) que expande verticalmente.

- **Grids:** colunas múltiplas viram coluna única (.profile-grid, .form-row).
- **Cards de vagas:** layout horizontal vira vertical.
- **Filtros:** inputs de busca ocupam 100% da largura.

O CSS usa variáveis CSS (**--primary**, **--radius**, **--shadow**) para consistência visual.

■ Dica: Mostre redimensionando o navegador ao vivo durante a defesa.

Pergunta 16 — O que é localStorage? Como usamos para autenticação?

localStorage é um armazenamento chave-valor no navegador, persistente entre abas e sessões do browser.

Após login bem-sucedido, salvamos os dados do usuário:

```
localStorage.setItem('tramos_user', JSON.stringify(userData))
```

Em cada página protegida, a função **requireLogin()** verifica se existe um usuário salvo:

```
function requireLogin() { const user = getUser(); if (!user) window.location.href = 'login.html'; return user; }
```

Ao fazer logout, o item é removido e o usuário é redirecionado para o login.

Limitação: localStorage não é o método mais seguro (vulnerável a XSS). Em produção, usaríamos tokens JWT com httpOnly cookies.

■ Dica: Abra o DevTools → Application → Local Storage durante a demo para mostrar os dados gravados.

6. FastAPI e Arquitetura do Backend

Pergunta 17 — O que é uma API REST? Como o projeto segue esse padrão?

API REST (Representational State Transfer) é um estilo arquitetural para comunicação entre sistemas via HTTP, usando verbos semânticos:

- **GET** — buscar dados (sem efeitos colaterais).
- **POST** — criar um novo recurso.
- **PUT** — atualizar um recurso existente.
- **DELETE** — remover um recurso.

No projeto, cada recurso tem sua URL (endpoint):

```
GET /usuarios → lista todos
```

```
POST /usuarios → cria novo
```

```
GET /usuarios/{id} → busca por ID
```

```
PUT /usuarios/{id} → atualiza
```

```
DELETE /usuarios/{id} → remove
```

As respostas são em **JSON** e os status HTTP corretos são usados (200, 201, 204, 400, 401, 404).

Pergunta 18 — O que é o CORS e por que precisamos configurá-lo?

CORS (Cross-Origin Resource Sharing) é uma política de segurança dos navegadores que impede uma página de fazer requisições para um domínio diferente do seu.

O frontend (aberto como arquivo local ou porta diferente) e o backend (porta 8000) têm origens diferentes. Sem CORS configurado, o browser bloquearia todas as requisições.

No projeto configuramos:

```
app.add_middleware(CORSMiddleware, allow_origins=['*'], ...)
```

allow_origins=['*'] permite qualquer origem — adequado para desenvolvimento. Em produção, substituiríamos por **['https://tramos.com.br']** para segurança.

■ Dica: CORS é verificado pelo NAVEGADOR, não pelo servidor. Uma requisição via curl/Postman nunca é bloqueada por CORS.

Pergunta 19 — O que são schemas Pydantic e para que servem?

Pydantic é a biblioteca de validação de dados do FastAPI. Os schemas definem a estrutura esperada dos dados de entrada e saída.

No projeto temos três tipos de schema para usuário:

- **UsuarioCreate** — dados recebidos no cadastro (nome, email, senha, tipo). A senha ainda é texto puro aqui.
- **UsuarioUpdate** — dados para atualização, todos opcionais (Optional).
- **UsuarioOut** — dados retornados nas respostas. **Não inclui a senha** — isso é fundamental para segurança.

O FastAPI valida automaticamente os dados recebidos contra o schema. Se um campo obrigatório faltar ou o email for inválido, retorna HTTP 422 automaticamente.

■ Dica: Diga: 'O UsuarioOut nunca retorna o campo senha, mesmo que exista no banco — isso é proteção por design.'

Pergunta 20 — Por que o código foi separado em pastas (routes, models, schemas)?

Essa organização segue o princípio de **Separação de Responsabilidades (SoC)**:

- **app/models.py** — define as tabelas do banco (camada de dados). Cada classe = uma tabela MySQL.
- **app/schemas.py** — define os contratos da API (o que entra e o que sai). Desacoplado do banco.
- **app/routes/** — lógica de negócio, uma rota por recurso (users.py, auth.py, jobs.py).
- **app/database.py** — configuração da conexão com o banco.
- **app/main.py** — ponto de entrada, registra as rotas e middlewares.

Isso facilita manutenção, testes e onboarding de novos desenvolvedores.

■ Dica: Mostre a estrutura de pastas no terminal: `ls -R tramos/backend/app/`

7. Busca, Categorias e Regex do Login

Pergunta 21 — Como funciona a busca de vagas no site? É feita no banco ou no frontend?

A busca é feita **100% no frontend (client-side)**, sem nova requisição ao banco.

O fluxo completo é:

1. Ao carregar a página, o JavaScript faz **GET /vagas** e salva tudo na variável **todasVagas**.
2. A cada letra digitada no campo de busca (evento **input**), a função **filtrar()** é chamada.
3. Ela percorre **todasVagas** e mantém apenas as que contêm o termo em **título**, **local** ou **descrição**:

```
(!termo || v.titulo.toLowerCase().includes(termo) ||  
v.local.toLowerCase().includes(termo) || v.descricao.toLowerCase().includes(termo))
```

4. O resultado filtrado é passado para **renderVagas()** que re-renderiza os cards na tela.

A busca é **case-insensitive** pois converte tudo para minúsculo com **.toLowerCase()**.

■ Dica: Vantagem: resposta instantânea sem latência de rede. Desvantagem: se houver milhares de vagas, carregar tudo de uma vez seria lento — nesse caso usaríamos busca no backend com query params.

Pergunta 22 — Como as categorias chegam no dropdown do filtro?

As categorias são carregadas dinamicamente do banco via API, não estão 'chumbadas' no HTML.

Fluxo:

1. **Backend:** A rota **GET /categorias** busca todas as linhas da tabela Categoria no MySQL e retorna JSON.
2. **Frontend (index.html):** A função **carregarCategorias()** faz fetch nessa rota e para cada categoria cria um **<option>** no select:

```
cats.forEach(c => { const o = document.createElement('option'); o.value =  
c.id_categoria; o.textContent = c.nome; sel.appendChild(o); });
```

3. **Filtro combinado:** O filtro de categoria verifica se o **id_categoria** da vaga bate com o valor selecionado no dropdown:

```
(!catId || String(v.id_categoria) === catId)
```

Isso significa que busca por texto e filtro por categoria funcionam **ao mesmo tempo** — o usuário pode digitar 'garçom' E selecionar 'Eventos' simultaneamente.

As 7 categorias no banco foram criadas pelo **seed.py**: Limpeza, Eventos, Construção, Tecnologia, Educação, Entregas, Design.

■ Dica: Mostre no Swagger: GET /categorias retorna o JSON. Depois mostre o dropdown na tela sendo preenchido com os mesmos dados.

Pergunta 23 — O login está usando regex correto para validar email e senha? Explique cada um.

Sim, os regex estão definidos em **auth.js** e são aplicados antes de qualquer requisição ao servidor.

Regex de email:

```
/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/
```


- `^` e `$` — ancora o início e fim da string (sem caracteres extras).
- `[a-zA-Z0-9._%+-]+` — parte local: letras, números e caracteres especiais permitidos.
- `@` — o arroba obrigatório.
- `[a-zA-Z0-9.-]+` — domínio: letras, números, pontos e hífens.
- `\.[a-zA-Z]{2,}` — extensão: ponto + mínimo 2 letras (.com, .br, .io).

Aceita: ana@tramos.dev ■ | Rejeita: ana@ ■, sem-arroba ■

Regex de senha:

```
/^.{6,}$/
```

- `.` — qualquer caractere (letra, número, símbolo).
- `{6,}` — mínimo 6 caracteres, sem máximo definido.

Aceita: senha123 ■ | Rejeita: 123 ■ (menos de 6)

Importante: o backend também valida via Pydantic (**EmailStr** para email). Se alguém bypassar o frontend (ex: via Postman), o backend ainda rejeita um email inválido com HTTP 422.

■ Dica: Durante a defesa, teste ao vivo: tente fazer login com 'email-sem-arroba' — o campo ficará vermelho instantaneamente, sem nem chamar o servidor.

Pergunta 24 — Por que o filtro usa `String(v.id_categoria) === catId` em vez de `==` diretamente?

Porque os tipos são diferentes: **v.id_categoria** é um número (Integer vindo do JSON), enquanto **catId** é uma string (valor de **select.value** no HTML, que sempre retorna string).

Em JavaScript, `===` (igualdade estrita) não faz conversão de tipo — então `1 === '1'` é **false**.

Para garantir a comparação correta, convertamos o número para string:

```
String(v.id_categoria) === catId // '1' === '1' → true ■
```

Alternativa seria usar `==` (igualdade frouxa), que faz coerção de tipo automaticamente, mas o uso de `===` é considerada boa prática em JavaScript pois evita comportamentos inesperados.

■ Dica: Esse é o tipo de detalhe técnico que demonstra autoria — mostra que você entende JavaScript tipado.

8. Perguntas Desafio — Para Pegar Autoria

Pergunta 25 — Se eu quisesse adicionar uma foto de perfil ao usuário, o que mudaria?

Precisaríamos alterar em três camadas:

1. Banco (models.py): Adicionar coluna na tabela Usuario:

```
foto_url = Column(String(500), nullable=True)
```

2. Schema (schemas.py): Adicionar o campo em UsuarioCreate, UsuarioUpdate e UsuarioOut.

3. Backend (routes/users.py): Tratar upload de arquivo com FastAPI (UploadFile) ou aceitar URL de imagem.

4. Frontend (profile.html): Adicionar input type='file' e exibir a imagem no avatar.

A forma mais simples seria aceitar uma URL de imagem (ex: do Imgur). Upload direto requeria salvar arquivo no servidor ou usar storage como AWS S3.

Pergunta 26 — O que aconteceria se dois usuários tentassem se cadastrar com o mesmo email ao mesmo tempo?

Temos **duas camadas de proteção** contra isso (race condition):

1. Verificação no código: A rota faz SELECT antes do INSERT. Mas em alta concorrência, dois requests poderiam passar essa verificação simultaneamente.

2. Constraint UNIQUE no banco: A coluna **email** tem UNIQUE no MySQL. O banco rejeitaria o segundo INSERT com um erro de duplicidade (IntegrityError).

Em produção, trataríamos o IntegrityError do SQLAlchemy e retornaríamos HTTP 409 Conflict.

A solução definitiva seria usar transações com **SELECT FOR UPDATE** ou confiar no UNIQUE do banco + tratamento de exceção.

Pergunta 27 — Como você testaria se a criptografia está funcionando corretamente?

De três formas:

1. Diretamente no MySQL: Após cadastrar, consultar a senha no banco e verificar que começa com \$2b\$ (prefixo bcrypt):

```
SELECT email, LEFT(senha,20) FROM Usuario;
```

2. Testando o login: A senha digitada pelo usuário deve funcionar no login, mesmo sendo diferente do hash gravado.

3. Verificando que hashes são únicos: Cadastrar dois usuários com a mesma senha e confirmar que os hashes são diferentes (por causa do salt).

Também podemos testar que senhas erradas retornam 401 e nunca 200.

■ Dica: Mostre isso ao vivo no terminal durante a defesa!

Pergunta 28 — Qual a diferença entre autenticação e autorização?

Autenticação (Authentication) = verificar **quem você é**. No projeto: o login verifica se email + senha estão corretos.

Autorização (Authorization) = verificar **o que você pode fazer**. No projeto: após logado, um freelancer não pode criar vagas (verificado no frontend via **requireTipo('empresa')**).

Analogia: autenticação é mostrar seu crachá na entrada; autorização é as permissões que esse crachá dá.

Em versões futuras, implementaríamos JWT tokens com claims de permissão para autorização no backend também.

Pergunta 29 — Se o banco cair, o que acontece com a aplicação?

O FastAPI tentará abrir uma conexão via SQLAlchemy no início de cada request. Se o banco estiver inacessível, o SQLAlchemy lançará um **OperationalError**.

O FastAPI retornará **HTTP 500 Internal Server Error** para o cliente.

No frontend, o bloco **catch** de cada fetch captura erros de rede e exibe um toast de erro ao usuário.

Em produção, trataríamos esse erro de forma mais elegante com exception handlers globais no FastAPI e connection pooling com retry automático.

9. Roteiro de Demo ao Vivo — Sprint 1

Siga este roteiro para demonstrar tudo em $\approx$ 3 minutos:

#	Ação	O que demonstra
1	Abrir <code>http://localhost:8000/docs</code>	Backend rodando + Swagger UI
2	POST <code>/usuarios</code> no Swagger (criar conta)	Create do CRUD
3	GET <code>/usuarios</code> no Swagger	Read (listagem)
4	PUT <code>/usuarios/{id}</code> no Swagger	Update do CRUD
5	Abrir <code>register.html</code> no browser	Interface com identidade visual
6	Preencher form com dados inválidos	Validação de campos
7	Cadastrar novo usuário corretamente	Create via frontend
8	Abrir DevTools → Application → Local Storage	Sessão armazenada
9	Redimensionar janela para <code>< 640px</code>	Responsividade
10	Ir para <code>profile.html</code> → editar nome	Update via frontend
11	Deletar conta → confirmar	Delete do CRUD
12	Tentar acessar <code>profile.html</code> sem login	Proteção de rota
13	Mostrar MySQL: <code>SELECT email, LEFT(session, 20) FROM usuario</code>	Senha 20 caracteres do usuário

Comando para iniciar o backend antes da defesa:

```
cd ~/Facul/XP-Criativa/trampos/backend
source venv/bin/activate
nohup uvicorn app.main:app --host 0.0.0.0 --port 8000 >> /tmp/uvicorn.log 2>&1 &
# Testar: curl http://localhost:8000/
```